

CSL

CryptoScript Language

A Domain-Specific Language for Cryptographic Programming

Authors	Avinash	(24CS10009)
	Jinansh Dalal	(24CS10016)
	Abhinav Tiwari	(24CS10077)
	Nithish Kumar	(24CS10106)
Repository	Software-Group-10/CryptoScript	
Version	1.0.0	
Language	Python 3.11+	

Contents

1	Introduction	3
1.1	Motivating Example	3
2	Project Structure	3
3	Installation & Setup	5
3.1	Prerequisites	5
3.2	Step 1 — Clone the Repository	5
3.3	Step 2 — Run the Setup Script	5
3.4	Step 3 — Verify the Installation	5
4	Running CryptoScript Programs	6
4.1	Compile and Run Immediately	6
4.2	Compile Only	6
4.3	Using the Compiler Directly from Python	6
5	Language Overview — The Compilation Pipeline	7
6	Data Types	7
6.1	Primitive Types	7
6.2	Parametric Types	7
6.2.1	ModInt[n]	7
6.2.2	Matrix[elem_type, rows, cols]	7
7	Basic Syntax	8
7.1	Variable Declaration	8
7.2	Assignment	8
7.3	Indentation	8
7.4	Operators	9
7.4.1	Arithmetic	9
7.4.2	Bitwise	9
7.4.3	Comparison & Logic	9
8	The Pipeline Operator	10
8.1	Semantics	10
8.2	Side-by-side Comparison	10
8.3	Pipelines inside Cipher Blocks	10
9	Built-in Functions	11
9.1	Cryptographic Transformations	11
9.1.1	Padding Schemes	11
9.2	Type Conversion Functions	12
10	S-Boxes and P-Boxes	12
10.1	Defining an S-Box	12
10.2	Defining a P-Box	13
10.3	Using S/P-Boxes in a Pipeline	13

10.4 Automatic Inverse	13
11 Cipher Definitions	13
11.1 Syntax	13
11.2 Complete Example	14
11.3 Using a Cipher	14
11.4 Generated Python Class	14
11.5 Automatic Inversion Rules	15
12 The repeat Construct	16
12.1 Inside a Cipher Pipeline	16
12.2 As a Standalone Statement	16
13 Control Flow	16
13.1 if / elif / else	16
13.2 for loops	17
13.3 while loops	17
13.4 match / case	17
13.5 break, continue, pass, return	17
14 Functions	17
15 Import Statements	18
16 A Complete Program	18
17 Testing	19
17.1 Running the Tests	19
17.2 Test Modules	20
17.3 Adding a New Test	20
18 Error Reference	21
18.1 LexerError	21
18.2 ParseError	21
18.3 CodeGenError	21
19 Token Reference	22
19.1 Literals	22
19.2 CryptoScript-specific Keywords	22
19.3 Data-type Keywords	22
19.4 Transform & Conversion Keywords	23
20 Grammar Summary	23
21 Frequently Asked Questions	24

1 Introduction

CryptoScript (CSL) is a domain-specific language purpose-built for writing cryptographic programs clearly and concisely. Cryptographic code written in general-purpose languages like Python is typically hard to read and error-prone, because it requires constant manual bookkeeping of data representations, modular arithmetic, and nested function calls.

CryptoScript eliminates this friction by providing:

- **Built-in cryptographic data types** — `Bytes`, `Hex`, `ModInt`, `Matrix`
- **A pipeline operator** (`|>`) for clean, left-to-right data transformations
- **Declarative cipher definitions** with *automatic decryption generation*
- **Python interoperability** — CSL compiles to plain Python, so it runs anywhere Python does

1.1 Motivating Example

Consider a substitution-xor cipher. Without CryptoScript:

```
1 result1 = permute(data, p)
2 result2 = substitute(result1, s)
3 result3 = xor(result2, key)
4 result_final = to_hex(result3)
```

Listing 1: Cryptographic code in raw Python

With CryptoScript:

```
1 result = data |> permute(p) |> substitute(s) |> xor(key) |>
  to_hex()
```

Listing 2: The same logic in CryptoScript

The pipeline version reads exactly like an English description of the cipher: “pad the data, XOR with IV, substitute, XOR with key, convert to hex”.

2 Project Structure

The repository is organised as follows. Understanding this layout will help you navigate the source, add tests, and extend the compiler.

3 Installation & Setup

3.1 Prerequisites

- Python **3.11** or later
- `git`
- A POSIX-compatible shell (`bash` / `zsh`)

3.2 Step 1 — Clone the Repository

```
git clone https://github.com/Software-Group-10/CryptoScript.git
cd CryptoScript
```

3.3 Step 2 — Run the Setup Script

The `scripts/setup.bash` script creates a virtual environment, installs all dependencies, and registers the shell aliases `run` and `compile` for you.

```
source scripts/setup.bash
```

Note

Always use `source` (not `bash`) so that the aliases defined inside the script are exported into your current shell session.

What the script does internally:

3.4 Step 3 — Verify the Installation

```
pytest tests/unit_tests/ -v
```

All tests should pass. A successful run looks like:

```
===== test session starts =====
collected 100 items

tests/unit_tests/test_tokens.py ..... PASSED
tests/unit_tests/test_lexer.py ..... PASSED
tests/unit_tests/test_parser.py ..... PASSED
tests/unit_tests/test_codegen.py ..... PASSED
===== 100 passed in 0.42s =====
```

4 Running CryptoScript Programs

After setup, you have two commands available.

4.1 Compile and Run Immediately

```
run myprogram.cs
```

This single command: `lexes myprogram.cs` → `parses` → `generates Python` → `executes` it. No intermediate files are left on disk.

4.2 Compile Only

```
compile myprogram.cs
```

This writes `myprogram.py` next to your `.cs` file. You can then inspect, modify, or execute the generated Python directly:

```
python myprogram.py
```

4.3 Using the Compiler Directly from Python

You can also invoke the compilation pipeline programmatically from your own Python code:

```
1 from cryptoscript.lexer import Lexer
2 from cryptoscript.parser import Parser
3 from cryptoscript.codegen import CodeGen
4
5 source = open("myprogram.cs").read()
6
7 tokens = Lexer(source).tokenize()    # Step 1    lex
8 ast    = Parser(tokens).parse()      # Step 2    parse
9 code   = CodeGen().generate(ast)     # Step 3    generate
10
11 print(code)                          # or exec(code)
```

Listing 3: Invoking the CryptoScript compiler from Python

Tip

Use `compile` when you want to audit or debug the generated Python. The generated file begins with a runtime preamble (helper functions for `xor`, `pad`, `ModInt`, etc.) followed by your translated code.

5 Language Overview — The Compilation Pipeline

CryptoScript source code goes through three stages before it runs:

`.cs` source $\xrightarrow{\text{Lexer}}$ Token stream $\xrightarrow{\text{Parser}}$ AST $\xrightarrow{\text{CodeGen}}$ `.py` source $\xrightarrow{\text{python}}$ Output

6 Data Types

6.1 Primitive Types

Type	Description	Example literal
<code>Int</code>	Arbitrary-precision integer	<code>42</code> , <code>-7</code>
<code>Bool</code>	Boolean value	<code>True</code> , <code>False</code>
<code>String</code>	UTF-8 text	<code>"hello"</code> , <code>'world'</code>
<code>Bytes</code>	Raw binary data	<code>b"\x41\x42"</code> , <code>b"ABC"</code>
<code>Hex</code>	Hexadecimal integer	<code>0xFF</code> , <code>0x1A3F</code>

6.2 Parametric Types

6.2.1 `ModInt[n]`

A modular integer that automatically reduces every arithmetic result modulo n .

```

1 n: ModInt[7] = 10      # stores 10 % 7 = 3
2 m: ModInt[7] = n + 5   # stores (3 + 5) % 7 = 1

```

Listing 4: `ModInt` example

Supported operations: `+`, `-`, `*`, `**` (all automatically modular). Internally implemented as a Python class `ModInt(value, modulus)`.

6.2.2 `Matrix[elem_type, rows, cols]`

A two-dimensional matrix whose elements are of the given type.

```

1 m: Matrix[Int, 3, 3] = ...   # 3x3 integer matrix

```

Listing 5: Matrix type annotation

Matrix objects support element-wise `+` and matrix `*` (multiplication).

7 Basic Syntax

7.1 Variable Declaration

Variables must be declared with an explicit type annotation before their first use. The syntax is:

```

                                name :  Type = value
1  x      :  Int           = 42
2  flag   :  Bool          = True
3  msg    :  String        = "hello, world"
4  data   :  Bytes         = b"\x01\x02\x03"
5  key    :  Hex           = 0xFF
6  n      :  ModInt[7]     = 10
7  mat    :  Matrix[Int, 2, 2] = [[1, 0], [0, 1]]

```

Listing 6: Variable declarations

Warning

Type names are **case-sensitive**. `Int` is a CryptoScript type; `int` is treated as a plain identifier. Similarly `Bool` $\neq$ `bool`.

7.2 Assignment

After declaration, variables can be reassigned without repeating the type:

```

1  x = 100
2  x += 5
3  x *= 2

```

All standard compound assignment operators are supported: `+=`, `-=`, `*=`, `/=`, `//=`, `%=`, `**=`, `&=`, `|=`, `^=`, `>=`, `<=`.

7.3 Indentation

Blocks are delimited by indentation, identical to Python. Four spaces per level is the recommended convention. Tabs are also accepted (counted as 4 spaces).

Warning

Mixing tabs and spaces within the same file will cause a `LexerError: NO MATCHING INDENTATION`. Be consistent.

7.4 Operators

7.4.1 Arithmetic

Operator	Meaning	Example
<code>+</code>	addition	<code>a + b</code>
<code>-</code>	subtraction	<code>a - b</code>
<code>*</code>	multiplication	<code>a * b</code>
<code>/</code>	true division	<code>a / b</code>
<code>//</code>	floor division	<code>a // b</code>
<code>%</code>	modulo	<code>a % b</code>
<code>**</code>	exponentiation	<code>a ** b</code>

7.4.2 Bitwise

Operator	Meaning	Example
<code>&</code>	bitwise AND	<code>a & b</code>
<code> </code>	bitwise OR	<code>a b</code>
<code>^</code>	bitwise XOR	<code>a ^ b</code>
<code>~</code>	bitwise NOT	<code>~a</code>
<code>«</code>	left shift	<code>a « 3</code>
<code>»</code>	right shift	<code>a » 3</code>

7.4.3 Comparison & Logic

Operator	Meaning
<code>==, !=</code>	equality / inequality
<code><, >, <=, >=</code>	relational
<code>and, or, not</code>	boolean logic
<code>in, is</code>	membership / identity

8 The Pipeline Operator

The pipeline operator `|>` is the central syntactic feature of CryptoScript. It threads the result of one expression as the *first argument* to the next function in a left-to-right chain.

8.1 Semantics

$$\begin{array}{lcl} x \mid> f(a, b) & \equiv & f(x, a, b) \\ x \mid> f(a) \mid> g(b) & \equiv & g(f(x, a), b) \end{array}$$

8.2 Side-by-side Comparison

```
1 result = to_hex(xor(substitute(xor(pad(data, "PKCS7"), iv), s
    ), key))
```

Listing 7: Without pipeline (hard to read)

```
1 result = data |> pad(PKCS7)
2           |> xor(iv)
3           |> substitute(s)
4           |> xor(key)
5           |> to_hex()
```

Listing 8: With pipeline (reads like prose)

The code generator lowers each pipeline into a series of simple Python assignments, making the generated code easy to inspect:

```
1 _result = data
2 _result = pad(_result, "PKCS7")
3 _result = xor(_result, iv)
4 _result = substitute(_result, s)
5 _result = xor(_result, key)
6 _result = to_hex(_result)
7 result  = _result
```

Listing 9: Generated Python for the pipeline above

8.3 Pipelines inside Cipher Blocks

Within a **cipher** definition the pipeline describes the entire encryption sequence (see Section 11).

9 Built-in Functions

All built-in functions are part of the *runtime preamble* injected at the top of every generated Python file.

9.1 Cryptographic Transformations

Function	Signature	Description
<code>substitute</code>	<code>(data, sbbox)</code>	Apply substitution box byte-by-byte. Each byte <code>b</code> of <code>data</code> is replaced by <code>sbox[b]</code> .
<code>permute</code>	<code>(data, pbox)</code>	Rearrange bytes of <code>data</code> according to index list <code>pbox</code> .
<code>xor</code>	<code>(data, key)</code>	XOR <code>data</code> with <code>key</code> (<code>key</code> is cyclically repeated if shorter than <code>data</code>).
<code>rotl</code>	<code>(val, n, bits=32)</code>	Rotate left by <code>n</code> bits within a <code>bits</code> -wide word. Works on both <code>Int</code> and <code>Bytes</code> .
<code>rotr</code>	<code>(val, n, bits=32)</code>	Rotate right by <code>n</code> bits.
<code>pad</code>	<code>(data, scheme)</code>	Pad <code>data</code> to a multiple of 16 bytes using the given scheme (<code>PKCS7</code> or <code>ZERO</code>).
<code>unpad</code>	<code>(data, scheme)</code>	Remove padding added by <code>pad</code> .

9.1.1 Padding Schemes

Scheme	Description
<code>PKCS7</code>	Pads with <code>p</code> bytes each with value <code>p</code> , where $p = 16 - (\text{len}(\text{data}) \bmod 16)$. Always adds at least one byte.
<code>ZERO</code>	Pads with <code>0x00</code> bytes until the length is a multiple of 16.

9.2 Type Conversion Functions

Function	Output type	Notes
<code>to_bytes</code>	<code>Bytes</code>	Converts <code>Int</code> , <code>Hex</code> , or <code>String</code> to bytes (big-endian by default).
<code>to_le_bytes</code>	<code>Bytes</code>	Little-endian variant.
<code>to_be_bytes</code>	<code>Bytes</code>	Explicit big-endian variant.
<code>to_hex</code>	<code>String</code>	Returns a "0x..." hex string.
<code>to_string</code>	<code>String</code>	UTF-8 decode for <code>Bytes</code> , <code>hex()</code> for <code>Int</code> .
<code>to_int</code>	<code>Int</code>	Big-endian byte interpretation; <code>int(s,0)</code> for strings.

Note

Type conversion functions **cannot be automatically inverted**. If your cipher pipeline contains a `to_hex` step, the auto-generated `decrypt` method will raise `NotImplementedError` at runtime. You must write a manual `def decrypt` in this case.

10 S-Boxes and P-Boxes

Substitution boxes (S-boxes) and permutation boxes (P-boxes) are named lookup-table constants that can be defined once at the top level and reused across cipher definitions.

10.1 Defining an S-Box

```
1 sbox my_s = [1, 0, 3, 2, 5, 4, 7, 6, 9, 8, 11, 10, 13, 12,
               15, 14]
```

Listing 10: S-box definition

The `sbox` keyword is followed by the name and an integer list. In the generated Python this becomes:

```
1 my_s = [1, 0, 3, 2, 5, 4, 7, 6, 9, 8, 11, 10, 13, 12, 15, 14]
```

10.2 Defining a P-Box

```
1 pbox my_p = [2, 0, 3, 1]
```

Listing 11: P-box definition

`my_p = [2, 0, 3, 1]` means: output byte 0 is input byte 2, output byte 1 is input byte 0, etc.

10.3 Using S/P-Boxes in a Pipeline

```
1 sbox s = [1, 0, 3, 2, 5, 4, 7, 6]
2 pbox p = [2, 0, 3, 1]
3
4 result = data |> substitute(s) |> permute(p)
```

10.4 Automatic Inverse

When `substitute` or `permute` appears inside a cipher’s encrypt pipeline, the code generator automatically computes and uses the inverse box for decryption:

```
1 # encrypt: substitute(_result, my_s)
2 # decrypt: substitute(_result, _cs_invert_sbox(my_s))
```

Listing 12: Auto-generated decrypt step

11 Cipher Definitions

The `cipher` block is the primary abstraction in CryptoScript. It defines a symmetric cipher as a named class with parameters.

11.1 Syntax

```
1 cipher CipherName(param1: Type1, param2: Type2, ...) ->
  Return Type:
2   encrypt:
3     data |> step1(...) |> step2(...) |> ...
```

Listing 13: Cipher definition syntax

- The cipher receives its **parameters** (key, IV, etc.) at construction time.
- The `encrypt` block contains a single pipeline expression starting from `data` — the implicit plaintext argument.

- The **decrypt** method is generated automatically by reversing and inverting each step.

11.2 Complete Example

```

1 sbox s = [1, 0, 3, 2, 5, 4, 7, 6]
2
3 cipher MyCipher(key: Bytes, iv: Bytes) -> Bytes:
4     encrypt:
5         data |> pad(PKCS7)
6             |> xor(iv)
7             |> repeat(3):
8                 xor(key) |> substitute(s)
9             |> xor(key)

```

Listing 14: A complete cipher definition

11.3 Using a Cipher

Once defined, a cipher is instantiated like a Python class and used with `.encrypt()` / `.decrypt()` methods:

```

1 mycipher = MyCipher(b"\xAA\xBB\xCC\xDD", b"\x55\x66\x77\x88"
2 )
3
4 plaintext = b"\x00\x01\x02\x03\x04\x05\x06\x07"
5
6 encrypted = mycipher.encrypt(plaintext)
7 decrypted = mycipher.decrypt(encrypted)
8
9 print("Plaintext :", plaintext)
10 print("Encrypted :", encrypted)
11 print("Decrypted :", decrypted)

```

Listing 15: Encrypting and decrypting with MyCipher

11.4 Generated Python Class

The code generator expands the above cipher definition into the following Python class:

```

1 class MyCipher:
2
3     def __init__(self, key: bytes, iv: bytes):
4         self.key = key
5         self.iv = iv
6
7     def encrypt(self, data):

```

```

8         _result = data
9         _result = pad(_result, "PKCS7")
10        _result = xor(_result, self.iv)
11        for _round in range(3):
12            _result = xor(_result, self.key)
13            _result = substitute(_result, s)
14        _result = xor(_result, self.key)
15        return _result
16
17    def decrypt(self, data):
18        _result = data
19        _result = xor(_result, self.key)                # inverse
20            of last xor
21        for _round in range(3):
22            _result = substitute(_result, _cs_invert_sbox(s))
23            _result = xor(_result, self.key)
24        _result = xor(_result, self.iv)
25        _result = unpad(_result, "PKCS7")
26        return _result

```

Listing 16: Auto-generated Python class for MyCipher

11.5 Automatic Inversion Rules

Encrypt step	Auto-generated decrypt step
xor(key)	xor(key) (XOR is its own inverse)
substitute(s)	substitute(_cs_invert_sbox(s))
permute(p)	permute(_cs_invert_pbox(p))
rotl(n)	rotr(n)
rotr(n)	rotl(n)
pad(PKCS7)	unpad(PKCS7)
unpad(PKCS7)	pad(PKCS7)
repeat(N): steps	repeat(N): inverted(steps)
to_hex, to_int, ...	Not invertible — raises NotImplementedError

12 The repeat Construct

The `repeat` keyword executes a pipeline block N times, equivalent to a loop of rounds in a block cipher.

12.1 Inside a Cipher Pipeline

```
1 cipher FeistelLike(key: Bytes) -> Bytes:
2     encrypt:
3         data |> repeat(16):
4             xor(key) |> substitute(s) |> permute(p)
```

Listing 17: repeat inside a cipher

12.2 As a Standalone Statement

`repeat` can also be used as a general-purpose loop outside cipher definitions:

```
1 repeat(10):
2     data = data |> xor(key) |> substitute(s)
```

Listing 18: repeat as a standalone loop

This is lowered to:

```
1 for _i in range(10):
2     _result = data
3     _result = xor(_result, key)
4     _result = substitute(_result, s)
5     data = _result
```

13 Control Flow

CryptoScript supports all standard Python control-flow constructs with identical syntax.

13.1 if / elif / else

```
1 if key == 0xFF:
2     mode = "max"
3 elif key > 0x80:
4     mode = "high"
5 else:
6     mode = "low"
```

13.2 for loops

```
1 for i in range(10):
2     data = data |> xor(key)
```

13.3 while loops

```
1 while flag:
2     data = data |> substitute(s)
3     flag = False
```

13.4 match / case

Pattern matching is supported with the same semantics as Python 3.10+:

```
1 match mode:
2     case "encrypt":
3         result = data |> xor(key)
4     case "decrypt":
5         result = data |> xor(key) # xor is its own inverse
6     case _:
7         result = data
```

13.5 break, continue, pass, return

All standard Python flow-control keywords are fully supported.

14 Functions

User-defined functions use the `def` keyword with typed parameters and an optional return type.

```
1 def apply_round(data: Bytes, key: Bytes, s: sbox) -> Bytes:
2     return data |> xor(key) |> substitute(s)
```

Listing 19: A user-defined helper function

Generated Python:

```
1 def apply_round(data: bytes, key: bytes, s) -> bytes:
2     _result = data
3     _result = xor(_result, key)
4     _result = substitute(_result, s)
5     return _result
```

Function calls can appear anywhere an expression is expected, including inside pipelines:

```
1 result = data |> apply_round(key, s) |> to_hex()
```

Warning

User-defined functions **cannot be automatically inverted**. If you include a call to a user-defined function inside a cipher's `encrypt` block, the auto-generated `decrypt` will raise `NotImplementedError`. Define `decrypt` manually, or restructure to use only built-in primitives.

15 Import Statements

CryptoScript supports Python-style imports that are passed through to the generated Python unchanged:

```
1 import os
2 from hashlib import sha256
```

This allows you to call Python standard library functions (e.g. `os.urandom(16)`) alongside CryptoScript primitives.

16 A Complete Program

The following end-to-end example demonstrates all major features: S-box definition, cipher definition with a repeat block, encryption, and decryption verification.

```
1 sbox s = [1, 0, 3, 2, 5, 4, 7, 6, 9, 8, 11, 10, 13, 12, 15,
2         14]
3 pbox p = [2, 0, 3, 1]
4
5 cipher TinyCipher(key: Bytes, iv: Bytes) -> Bytes:
6     encrypt:
7         data |> pad(PKCS7)
8             |> xor(iv)
9             |> repeat(4):
10                 xor(key) |> substitute(s) |> permute(p)
11             |> xor(key)
12
13 key      : Bytes = b"\xDE\xAD\xBE\xEF"
```

```
14 iv          : Bytes = b"\xCA\xFE\xBA\xBE"
15 plaintext    : Bytes = b"\x00\x01\x02\x03\x04\x05\x06\x07"
16
17 cipher_obj = TinyCipher(key, iv)
18
19 encrypted = cipher_obj.encrypt(plaintext)
20 decrypted = cipher_obj.decrypt(encrypted)
21
22 print("Plaintext :", plaintext)
23 print("Encrypted :", encrypted)
24 print("Decrypted :", decrypted)
25 print("Match      :", plaintext == decrypted)
```

Listing 20: example.cs — a complete CryptoScript program

Save this as `example.cs` and run:

```
run example.cs
```

Expected output:

```
Plaintext : b'\x00\x01\x02\x03\x04\x05\x06\x07'
Encrypted : b'...' (16 bytes of cipher text)
Decrypted : b'\x00\x01\x02\x03\x04\x05\x06\x07'
Match      : True
```

17 Testing

The test suite lives in `tests/unit_tests/` and covers all four compiler stages.

17.1 Running the Tests

```
# Run all tests
pytest tests/unit_tests/ -v

# Run a single test module
pytest tests/unit_tests/test_lexer.py -v

# Run tests matching a name pattern
pytest tests/unit_tests/ -k "hex" -v
```

17.2 Test Modules

Module	What it tests
<code>test_tokens.py</code>	KEYWORDS lookup table completeness, <code>Token</code> constructor, <code>__str__</code> , and <code>__eq__</code>
<code>test_lexer.py</code>	Identifiers, keywords, integers, hex literals, byte strings, regular strings, all operators and delimiters, indentation (INDENT/DE- DENT), multi-line programs, edge cases and <code>LexerError</code> conditions
<code>test_parser.py</code>	Variable declarations, assignments, expressions, pipelines, if/- for/while/match/with statements, cipher/sbox/pbox definitions, function definitions, import statements, parse errors
<code>test_codegen.py</code>	Full round-trip: CSL source → compiled Python → executed output matches expected result

17.3 Adding a New Test

Tests follow the standard `pytest` convention. Here is an annotated example:

```

1 import pytest
2 from cryptoscript.lexer import Lexer, LexerError
3 from cryptoscript.tokens import TokenType
4
5 def lex(src):
6     return Lexer(src).tokenize()
7
8 def types(tokens):
9     return [t.type for t in tokens
10             if t.type not in (TokenType.NEWLINE, TokenType.
11                               EOF)]
12
13 def test_pipeline_operator():
14     """The |> operator must lex as a single PIPELINE token."""
15     assert types(lex("|>")) == [TokenType.PIPELINE]
16
17 def test_invalid_hex_raises():
18     """0x with no hex digits must raise LexerError."""
19     with pytest.raises(LexerError):
20         lex("0x")

```

Listing 21: Example test for the lexer

18 Error Reference

18.1 LexerError

Raised by the lexer when the input contains an invalid character sequence. Format:

```
LexerError on line N: <message>
```

Common causes and messages:

Message	Cause
INVALID HEX	0x not followed by any hex digit
INVALID HEX LITERAL	0x1G — non-hex character after digits
INVALID INT LITERAL	248ab — letter immediately after digits
INVALID STRING	Unterminated string or unknown escape sequence
INVALID BYTESTRING	Unterminated bytestring or invalid \xHH escape
NO MATCHING INDENTATION	Dedent to a level that was never an indent level
unexpected character 'X'	Character X (e.g. @) is not part of the language
UNEXPECTED EOF	Source ends in the middle of a string literal

18.2 ParseError

Raised by the parser when the token stream does not match the grammar. Format:

```
ParseError on line N: EXPECTED <token>, GOT <token> (<value>)
```

18.3 CodeGenError

Raised during code generation for unsupported constructs. Format:

```
CodeGen Error : <message>
```

Common cause: a type-conversion function (`to_hex`, etc.) or a user-defined function appearing inside a cipher pipeline, making automatic decryption impossible.

19 Token Reference

This section provides the complete list of token types recognised by the lexer.

19.1 Literals

Token	Example	Description
INT_LIT	42, 0	Integer literal
HEX_LIT	0xFF, 0X1A	Hexadecimal literal
BYTES_LIT	b"\x41"	Byte string literal
STRING_LIT	"hello"	String literal

19.2 CryptoScript-specific Keywords

Keyword	Token	Purpose
cipher	CIPHER	Cipher block definition
encrypt	ENCRYPT	Encryption sub-block
sbox	SBOX	Substitution box definition
pbox	PBOX	Permutation box definition
repeat	REPEAT	Round-repeat construct
>	PIPELINE	Pipeline operator

19.3 Data-type Keywords

Keyword	Token
Int	INT
Bool	BOOL
String	STRING
Bytes	BYTES
Hex	HEX
ModInt	MODINT
Matrix	MATRIX

19.4 Transform & Conversion Keywords

Keyword	Token	Keyword	Token
substitute	SUBSTITUTE	to_bytes	TO_BYTES
permute	PERMUTE	to_le_bytes	TO_LE_BYTES
xor	XOR	to_be_bytes	TO_BE_BYTES
rotl	ROTL	to_hex	TO_HEX
rotr	ROTR	to_string	TO_STRING
pad	PAD	to_int	TO_INT
unpad	UNPAD		

20 Grammar Summary

The following is an informal EBNF-style grammar for CryptoScript.

```

program      ::= statement* EOF

statement    ::= import_stmt
              | from_import_stmt
              | func_def
              | cipher_def
              | sbbox_def
              | pbox_def
              | if_stmt
              | while_stmt
              | for_stmt
              | match_stmt
              | with_stmt
              | return_stmt
              | repeat_stmt
              | break | continue | pass
              | assign_or_expr

var_decl     ::= IDENT ':' type ['=' expr] NEWLINE
assign       ::= expr assign_op expr NEWLINE
assign_op    ::= '=' | '+=' | '-=' | '*=' | '//=' | ...

sbbox_def    ::= 'sbbox' IDENT '=' '[' int_list ']'

```

```

pbox_def      ::= 'pbox' IDENT '=' '[' int_list ']'

cipher_def    ::= 'cipher' IDENT '(' params ')' '->' type ':'
               NEWLINE
               INDENT encrypt_block DEDENT
encrypt_block ::= 'encrypt' ':' NEWLINE INDENT pipeline
               DEDENT

pipeline      ::= expr ('|>' pipeline_step)*
pipeline_step ::= func_call
               | 'repeat' '(' expr ')' ':' NEWLINE INDENT
               pipeline DEDENT

func_def      ::= 'def' IDENT '(' params ')' ['->' type] ':'
               block

type          ::= 'Int' | 'Bool' | 'String' | 'Bytes' | 'Hex'
               | 'ModInt' '[' INT_LIT ']'
               | 'Matrix' '[' type ',' INT_LIT ',' INT_LIT
               ']'

expr          ::= pipeline_expr | or_expr
or_expr       ::= and_expr ('or' and_expr)*
and_expr      ::= not_expr ('and' not_expr)*
not_expr      ::= 'not' not_expr | comparison
comparison    ::= bitwise_or (comp_op bitwise_or)*

```

21 Frequently Asked Questions

Q: Why do I get `NotImplementedError` when calling `.decrypt()`? Your cipher pipeline contains a step (typically a type-conversion like `to_hex` or a user-defined function) that cannot be automatically inverted. Remove it from the pipeline or write `decrypt` manually.

Q: Can I use Python libraries inside a `.cs` file? Yes. Add an `import` statement at the top of your file. The generated Python will include it verbatim, and you can call the imported names freely.

Q: How do I inspect the generated Python? Use the `compile` command instead of `run`. This writes a `.py` file that you can open in any editor.

Q: Is the generated Python standalone? Yes. The `.py` file begins with a self-contained runtime preamble and has no dependency on the `cryptoscript` package at execution time.

Q: Can I define multiple ciphers in one file? Yes. Each `cipher` block generates an independent Python class, and they can reference the same S-boxes and P-boxes.